

Deliverable 1

SENG3011 22T1

Design Details

Team f0b5 – Matthew Brian, Jaeff Hong, Mathieu Paturel, Nicholas Ramsay and Huiyi Wang

(z5330484, z5316653, z5312157, z5367013, z5261844)

Contents

Question One.....3

 API Design.....3

 API Structure3

 API Formatting3

Question Two..... 4

 Parameters and Examples 4

 Parameters.....6

 Sample Requests with Successful Responses6

 Return Codes and Info 8

Question Three9

 Server9

 Database.....10

 Scraping10

 Deployment Environment.....10

 Testing.....10

 Development.....11

References.....12

Question One

Describe how you intend to develop the API module and provide the ability to run it in web service mode

API Design

The API will be created using the REST convention. We intend to develop our API using a Python3 Flask server connected to an SQL database, accessible via a nginx proxy. Nginx allows us to make performant asynchronous requests to the flask server in a production environment. The database will be periodically updated by our article parser. All the above will be hosted on, and accessible via, [Linode](#).

The API is accessible via port 80 at the site <http://seng3011.duckdns.org> and *takes REST requests as described in Q2* (not all of which are implemented yet). DuckDNS is a DNS provider which forwards all requests to the above URL on to the Linode server at 172.105.183.203. The path <http://seng3011.duckdns.org/alive> is currently available with a sample 'health check' request.

API Structure

The living API design documentation is being constantly updated. Sample http requests and their corresponding parameters, as well as response codes and examples can be found here. It is currently accessible at https://app.swaggerhub.com/apis/tanyawhy/SENG3011_f0b5/1.0.0#/.

Since this application retrieves disease reports and articles from the CIDRAP website, only GET methods will be implemented. Since the specification may be changed at a later stage, we will update the Swagger documentation during the project to keep up to date with requirements. The built-in versioning support of Swagger will be employed to preserve the change history while maintaining a clean published version.

API Formatting

There are four endpoints provided by our application, '/alive', '/article/filter', '/report/filter' and '/report /from_article_url'. All endpoints, except '/alive', follow the resource-oriented naming convention and are formatted as singular form of the object class to fetch followed by the search criteria.

Currently the provided specification only requires a small amount of data to be queryable. This is why there are only a small number of endpoints, but it also justifies why the 'filter' suffix is appended to two of the endpoints despite seeming redundant. If more data was made queryable due to a changing spec, we could add it easily under an appropriate categorisation without confusing the current endpoints. For example, an endpoint to *POST* an article or report to the database could be added under 'article/new', or 'report/new'.

Question Two

Discuss your current thinking about how parameters can be passed to your module and how results are collected. Show an example of a possible interaction (e.g. sample HTTP calls with URL and parameters)

Parameters and Examples

Naturally, users are expected to filter articles and reports with `/article/filter` and `/report/filter`. These endpoints will provide great freedom in input parameter choices.

`/article/filter` retrieves all articles which match the input key terms and locations. It will also filter search within a specific time range. The same filtering functionality is achieved for disease reports via `/report/filter`.

Date parameters, namely `start_date` and `end_date`, should follow the format `yyyy-MM-ddTHH:mm:ss`, e.g., `2015-10-01T08:45:10`. Every variable except year of the given date parameter is omissible using `x`. For example, `2022-xx-xxTxx:xx:xx`.

Key search terms, `key_term` in the API, can be anything as specified in appendix B of the spec as required. For example:

Key Term	
General	Specific
Outbreak	Zika
Infection	MERS
...	...

Several key terms can be concatenated in one request, separated by commas. As expected, this will retrieve all results with any of the key terms input. For example, `/article/filter?key_term=fever,infection` returns all articles that mentioned the symptoms of fever or infection.

On the other hand, search by location allows no more than one geographic location. Locations will be queried using actual location names, such as `'Sydney'`, instead of geographic identifiers (GEOID) since expressing location in words is more intuitive for the users and saves them from unnecessary geolocation lookup. All query parameters are case-insensitive.

Using only two endpoints to cover the filtering/search may result in an imbalance in traffic amongst endpoints in this service. However, this provides users much more freedom to customise search criteria.

Users will be able to choose the number of filters in the request by adding or deleting parameters. For example, specifying key terms and location without time constraint in article data filtering, using `/article/filter?key_terms=fever&location=Sydney`, returns all articles about fever in Sydney. Moreover, if no filter is provided in the request to `/article/filter`, all articles on the CIDRAP website will be retrieved. This also applies to `/report/filter` in the same manner.

GET /article/filter

Retrieve articles by filtering

Parameters Try it out

Name	Description
start_date	news issued after this date will be returned
string (query)	Example : 2021-10-01T08:45:10
	2021-10-01T08:45:10
end_date	news issued before this date will be returned
string (query)	Example : 2021-11-08T19:37:12
	2021-11-08T19:37:12
key_terms	key search terms
string (query)	Example : Fever
	Fever
location	geographic location
string (query)	Example : Netherlands
	Netherlands

Please go to:

https://app.swaggerhub.com/apis/tanyawhy/SENG3011_f0b5/1.0.0#/

to see the full documentation with more details and specifics like return codes!

Responses

Code	Description	Links
200	OK	No links
	Media type: application/json Controls: Accepts: header Example Value Schema <pre> e; increased antibiotics for sepsis; Jamestown C anyon cases; Zika in India; H5N1 avian flu outbr eaks", "reports": [{ "diseases": ["influenza a/h5n1"], "syndromes": ["Fever of unknown Origin"], "event_date": "2021-11-05 xx:xx:xx", "locations": [{ "country": "Netherlands", "location": "North Holland Province" }] }] } </pre>	
400	Bad Request	No links
400	Bad Request	No links
	Media type: application/json Example Value Schema <pre> { "message": "start_date cannot be later than end_ date" } </pre>	

The `/report/from_article_url` API was created so that users can access disease reports linked to a specific article. They can do that simply by passing the article's URL to this endpoint.

Furthermore, we have an endpoint to ensure the service is alive to aid debugging efforts in the event of service downtime. This endpoint can also help us to monitor the server performance, including the connection status of scraper and database.

Parameters

For all three endpoints above, query parameters are used to retrieve the reports and articles, as shown in sample Swagger calls. These all follow the GET REST convention. Note that although placing parameters in URL is also acceptable for GET methods, it usually applies to the circumstance where users know the unique identifier, such as id. Thus, we decided that query string is the best option for passing parameters to our endpoints.

Sample Requests with Successful Responses

URL with Query Parameters	Response
http://seng3011.duckdns.org/article/filter?start_date=2021-10-01T08%3A45%3A10&end_date=2021-11-08T19%3A37%3A12&key_terms=Fever&location=Netherlands	<pre>[{ "url": "https://www.cidrap.umn.edu/news-perspective/2021/11/news-scan-nov-08-2021", "date_of_publication": "2021-11-08 xx:xx:xx", "headline": "News Scan for Nov 08, 2021", "main_text": "COVID and rheumatologic disease; Increased antibiotics for sepsis; Jamestown Canyon cases; Zika in India; H5N1 avian flu outbreaks", "reports": [{ "diseases": ["influenza a/h5n1"], "syndromes": ["Fever of unknown Origin"], "event_date": "2021-11-05 xx:xx:xx", "locations": [{ "country": "Netherlands", "location": "North Holland Province" }] }] }]</pre>
http://seng3011.duckdns.org/article/filter?start_date=2021-10-01Txx%3Axx%3Axx&end_date=2021-11-08Txx%3Axx%3Axx&key_terms=Fever	
http://seng3011.duckdns.org/article/filter?end_date=2021-11-08T19%3A37%3A12&key_terms=Fever%2CInfection&location=Netherlands	
http://seng3011.duckdns.org/article/filter?key_terms=Fever%2Cinfection&location=Netherlands	
http://seng3011.duckdns.org/article/filter?location=Netherlands	

http://seng3011.duckdns.org/report/filter?start_date=2021-10-01T08%3A45%3A10&end_date=2021-11-08T19%3A37%3A12&key_terms=Fever&location=Netherlands	[
http://seng3011.duckdns.org/report/filter?start_date=2021-10-01T08%3A45%3A10&end_date=2021-11-08T19%3A37%3A12&key_terms=Fever	{
http://seng3011.duckdns.org/report/filter?end_date=2021-11-08T19%3A37%3A12&key_terms=Fever&location=Netherlands	"diseases": [
http://seng3011.duckdns.org/report/filter?key_terms=Fever%2Cinfection&location=Netherlands	"influenza a/h5n1"
http://seng3011.duckdns.org/report/filter?location=Netherlands	],
	"syndromes": [
	"Fever of unknown Origin"
	],
	"event_date": "2021-11-05 xx:xx:xx",
	"locations": [
	{
	"country": "Netherlands",
	"location": "North Holland Province"
	}
	]
	}
	]
http://seng3011.duckdns.org/report/from_article_url?url=https%3A%2F%2Fwww.cidrap.umn.edu%2Fnews-perspective%2F2021%2F11%2Fnews-scan-nov-08-2021	[
	{
	"diseases": [
	"influenza a/h5n1"
	],
	"syndromes": [
	"Fever of unknown Origin"
	],
	"event_date": "2021-11-05 xx:xx:xx",
	"locations": [
	{
	"country": "Netherlands",
	"location": "North Holland Province"
	}
	]
	}
	]
http://seng3011.duckdns.org/alive	{
	"scrapy_online": true,
	"sql_connected": true
	}

This data has been copy/pasted from swagger for your convenience! If you want to see it properly formatted go to the website as linked previously

Return Codes and Info

The endpoints will return status code 200 with the filtered reports or articles in a JSON array, following the standard transmit format in SENG3011 project spec.

Specifically, *article* data contains a URL, published date, headline, main text and a list of associated reports of the article. *Report* data contains lists of diseases, syndromes and locations mentioned, as well as a date on which the case occurred. We chose the array format for the response since it is likely to have multiple results on the CIDRAP website that fit the filtering criteria.

Requests with malformed query parameters will result in a 400 *bad request* error with error messages shown in the table below.

Endpoint	Error Scenario	Code	Response
article/filter report/filter	Start date is later than end date	400	{ "message": "start_date cannot be later than end_date" }
	Start date is later than current date	400	{ "message": " start_date cannot be later than today" }
	Location does not exist	400	{ "message": " Invalid location" }
	Key term does not exist in Appendix B in project specification	400	{ "message": "Unknown key_term" }
/report/from_article_url	Malformed URL or URL does not exist	400	{ "message": "Invalid url" }

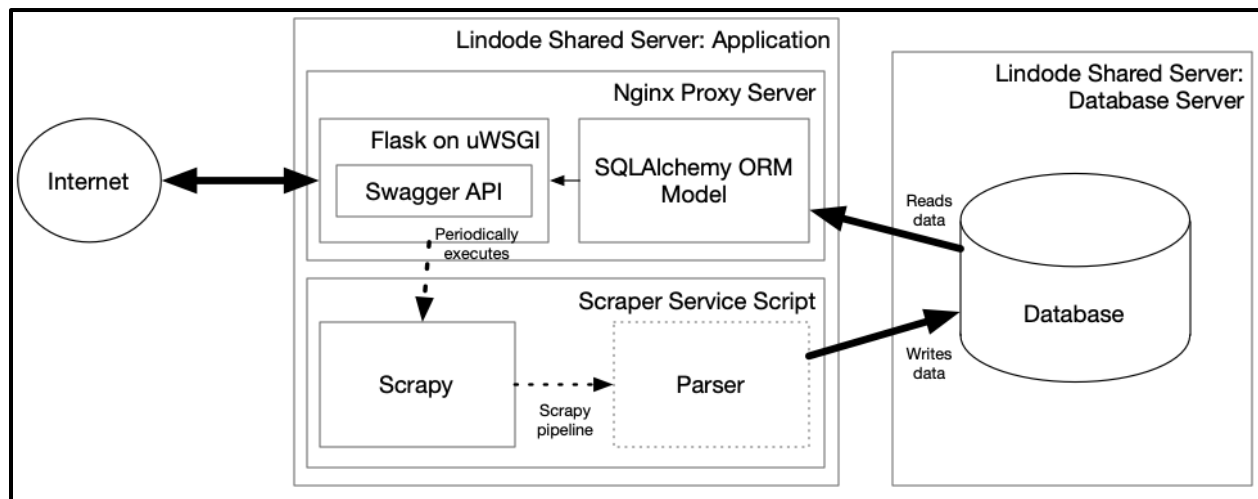
Question Three begins on the next page

Question Three

Present and justify implementation language, development and deployment environment and specific libraries that you plan to use

Our implementation is built on **Python** and **SQL**, respectively composing our application and database services.

Our application service, or our API, is produced with **Swagger** on a **Flask** server alongside a **Scrapy** microservice tasked with the scraping and collecting CIDRAP data. This is deployed on a **Linode** shared server. On the Linode server we are running a **nginx** proxy server that points to an **uWSGI** application server.



Server

Only Flask will be used in local development as Flask offers a flexible debugging and development mode for use on local machines. uWSGI and Nginx are used merely for their security and performance enhancements as Flask in its lonesome isn't designed for deployment into a production setting.

We chose Python as our main language for three reasons. Firstly, all team members have had extensive experience with it^[4] at university in COMP1531 and SENG2021 to develop a flask API. Secondly, Python has extensive third-party libraries that integrate closely with it, allowing connections to SQL databases. Thirdly, its widespread adoption will make it easier to maintain, and update with new features in the future.

We have also found Swagger integrates well with Flask to automate and ease API documentation. So far Swagger has been easy to learn, and our team is excited to continue using it.

That said, we made sure to compare Python and other candidate languages. Being interpreted, Python lacks the performance of many other API creation languages, especially modern performance-oriented languages like Go^[1]. Unfortunately, Go – our second pick – would require three of our team members to learn it from scratch, unviable for our need for a quick development solution.

The only viable alternatives, **Node**, **ExpressJS**, **Java** with **Spring** and Python with **Django** all suffer from the same primary issue, our lack of experience which mitigates their advantages. This is despite Node and Spring offering better API response times in performance tests ^[2] (besides Django REST framework ^[3]). It is vital that our team can begin work on our project as soon as possible without being slowed down by learning. As a team, we have opted to choose ease of development over performance.

Database

When it comes to our database, we will use a SQL server running on the same Linode server. The data we are required to store is outlined in the project spec, and it seems using a relational database will be apt. As in previous decisions, a lack of comprehensive team knowledge of non-relational databases pushed us towards using SQL instead of an alternative like MongoDB. Furthermore, a relational database seems to be better suited to our well-defined data storage requirements.

Our team has decided upon using other third-party libraries to assist our development. For our API, we will be using **SQLAlchemy** to manage database interactions. SQLAlchemy acts as an ORM (object-relational mapping) tool. This will assist us in the development of our model to manage database communication more easily. It is essential that our API uses a model to separate our endpoints from the database structure. Additionally, SQLAlchemy will protect against bad requests (like with SQL injection) and generally reduce code complexity. Out of all available python ORM packages, we chose SQLAlchemy due to its tight integration with Flask.

Scraping

We also had to choose a package to help fill the database. On a recommendation from course admin, we investigated the Scrapy package and liked its design. Specifically, we liked its ability to easily pipeline data to sources we specified. Scrapy will be used to collect raw data and do some basic first-order processing to separate it out into respective database fields.

Bundled with Scrapy will be a parser process responsible for reading page content and collating report data. As Flask and Scrapy are python packages, this will likely be written in python, but no specifics have been decided on yet. The parser will act as its own black-box process to reduce coupling between system components. Therefore, Scrapy's ability to pipeline data is ideal for our agile development approach.

Deployment Environment

Our API code will be deployed on a Linux server as this is the environment we have most experience with due to uni. The packages we require are also simpler to install and link together on Linux.

Testing

We plan to use the **Pytest** module to create tests for our server endpoints (integration testing), components of our scraper and wherever else we need assurance that such testing will provide. These tests can be executed locally, however, they will also be implemented in a **GitHub Action** to be executed automatically.

For our testing suite, we considered alternatives to Pytest, namely **unittest** which conveniently comes bundled with Python. Unittest provided all the testing functionality we need but requires boilerplate testing classes to be manually written. Pytest on the other hand provides most of the testing template, we just need to write the test cases. Most importantly, Pytest is the most closely integrated testing library for Flask applications ^[7] and as such, it is well integrated and documented for our purposes.

For our testing suite, we considered alternatives to Pytest, namely **unittest** which conveniently comes bundled with Python. Unittest provided all the testing functionality we need but requires boilerplate testing classes to be manually written. Pytest on the other hand provides most of the testing template, we just need to write the test cases. Most importantly, Pytest is the officially recommended testing suite for Flask applications ^[7] and as such, its use as a Flask endpoint testing tool is well integrated and documented. Other testing suites besides unittest were considered (particularly DocTest, Nose2 and Testify), but they mostly focus on extending testing functionality beyond the scope of our requirements.

Most significantly, Pytest can easily be executed in a diverse range of environments, locally, or as a part of a continuous integration system (there is existing support for Pytest GitHub actions ^[6]). This means no code that fails our test cases can be pushed to our repository. If a push to our repository does fail, the GitHub action will then return valuable information describing the failed test case. This contrasts with unittest which requires more code to be written to replicate this versatility.

Development

Lastly, and for the sake of code cleanliness, we have employed the **Black** linter to enforce a strict yet consistent style on our code. This is done to remove the stress of consistently formatting our python code. This linting procedure is done during pushes to our GitHub repository and can be set up to run automatically on a local machine. This will enforce a more maintainable and consistent codebase styling.

References

- [1] <https://medium.com/innovation-incubator/rest-api-performance-comparison-python-vs-golang-dc4decbd0543>
- [2] <https://www.techempower.com/benchmarks/>
- [3] <https://www.grandmetric.com/2020/08/10/python-rest-frameworks-performance-comparison/>
- [4] <https://flask.palletsprojects.com/en/2.0.x/tutorial/deploy/>
- [5] <https://docs.pytest.org/en/7.0.x/>
- [6] <https://github.com/marketplace/actions/actions-pytest>